DAA Assignment

Ques 1 Explain Brute force Algorithm design technique. Illustrate it by solving a small instance of the Traveling Salesman Problem (TSP) & analyze its time Complexity.

Sol Brute force is straightforward approach that tries all possible solutions to find optimal one. It doesnot use any optimization strategy & checks every possible combination / permutation.

→ Application to Travelling Salesman problem (TSP)

Problem:- Given cities & distances b/w them, find the shortest possible route that visits each city exactly once & returns to starting city.

Example:- Let there be 4 cities: A, B, C, D

	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

All possible Tours starting from A

$A \to B \to C \to D \to A = 10 + 35 + 30 + 20 = 95$

$A \to B \to D \to C \to A = 10 + 25 + 30 + 15 = 80$

$A \to C \to B \to D \to A = 15 + 35 + 25 + 20 = 95$

$A \to C \to D \to B \to A = 15 + 30 + 25 + 10 = 80$

$A \to D \to B \to C \to A = 20 + 25 + 35 + 15 = 95$

$A \to D \to C \to B \to A = 20 + 30 + 35 + 10 = 95$

Minimum Cost = 80

Optimal path $A \to B \to D \to C \to A$ /
$A \to C \to D \to B \to A$

Time Complexity for n cities $= (n-1)!$ $T(n) = O(n!)$

Ques 2 Describe Greedy algorithm paradigm. Explain why Greedy works for fractional Knapsack but fails for 0/1 Knapsack.

Ans It builds the solution step by step, always choosing the locally optimal choice, hoping to reach global optimum.

→ Makes best choice at each step
→ Simple & fast

⊙ Fractional Knapsack :- fractions of Items are allowed
Strategy - Select items based on maximum profit
⊙ why greedy works → Items can be broken into fractions
choosing highest ratio always improves profit

Time Complexity $O(n \log n)$

0/1 Knapsack :- either take item fully or not all.
why greedy fails - local optimal choice may block better global solution.

Example:

Item	Weight	Profit	P/W
A	10	60	6
B	20	100	5
C	30	120	4

Capacity = 50
Greedy choice A+B → profit = 160
Optimal B+C → profit = 220
Hence, Greedy fails

Ques3 Explain (DP). formulate DP sol^n for 0/1 Knapsack including Optimal Substructure, recurrence, rel^n & complexity

Sol DP → solves problems by breaking them into Overlapping Subproblems, solving each once & storing results

Key properties ⊙ Optimal Substructure
⊙ Overlapping Subproblems

0/1 Knapsack DP formulation →

n = no. of items wt [i] = weight of item i
W = capacity val [i] = value of item i

$DP_W[i][w]$ = maximum profit using first i items with capacity

Optimal Substructure :- Optimal sol^n of problem depends on optimal sol^n of smaller subproblems.

Recurrence Rel^n :- $DP[i][w] = \begin{cases} DP[i-1][w], \\ \max(DP[i-1][w], val[i] + DP[i-1][w - wt[i]]) \end{cases}$

Time & Space Complexity : Time : $O(nW)$
Space : $O(nW)$

Ques4 what is Backtracking? Demonstrate how it is used to solve TSP / Bin Packing. Explain State space tree & pruning.

Ans Backtracking is technique that builds sol^n incrementally & abandoms (backtracks) a sol^n as soon as it determines it cannot lead to valid sol^n.

State Space Tree :-

A tree represential all possible choices during problem solving.

Pruning :- cutting off branches that cannot give better sol^n, thus reducing search space.

→ Backtracking in TSP ⊙ Build path city by city
⊙ If partial path cost > current minimum → prune that path

Steps :- Start from city 1
Add next unvisited city
If cost exceeds minimum → stop
Continue until all cities visited

Ques 5 Explain Branch & Bound. Compare Branch & bound & backtracking in terms of search space, efficiency, applicability. Illustrate using knapsack 0/1

Sol Branch & Bound optimization technique that eliminates larger part of search space using upper & lower bound.

Feature	Backtracking	Branch & Bound
Search Space	DFS traversal	Best first / BFS
Pruning	Based on feasibility	Based on Bounds
Efficiency	Less efficient	More efficient
Best for	Decision problems	Optimization problems.

Application Example : TSP
⊙ Backtracking : eliminates path exceeding cost
⊙ Branch & Bound : calculates lower bound of partial path & prunes if it exceeds best sol^n.